

LayerForge: Deterministic Layer Decomposition for AI Output Convergence –An Empirical Report on Newman Modularity Outperforming CPM in Small-N Text Domains

Style: 報告 framing –「これを我々は主張する」ではなく「これが成立することが分かった」。reader time cost > return という publication 哲学に従い、過剰な claim を避け、再現可能な observations を中核に据える。License: MIT (本リポジトリ全体と同じ) Reproducibility: 全測定の生 CSV / scripts / JSON は scripts/k_sweep/ 配下に同梱、commit hash で固定。

Abstract

Conversational AI output tends to include content beyond what the questioner is actually concerned with. We report on **LayerForge**, a deterministic pipeline that decomposes a corpus into 4 ± 1 thematic layers and returns only the layer(s) relevant to a query. The pipeline combines five established components –Cowan's working-memory upper bound, Newman modularity for community detection, SCA-style per-layer distillation, HERCULES-style hierarchical KMeans, and a deterministic core with no LLM in the analysis loop –chosen so that "AI noise filtering" does not itself depend on an AI. We re-implemented a CPM (Constant Potts Model) backend (MIT-pure, self-implemented) to test whether the theoretically resolution-limit-free alternative would outperform Newman in this small-N text domain (operational target $N=20-40$, external scaling check up to $N=100$ on 20 Newsgroups). Across 119-row dual-method correlation sweep, 196-cell $N\times K$ heatmap, 95-row ARI/NMI comparison, and the external 20-Newsgroups benchmark, **the Newman backend exceeded CPM by a large effect size in our test range** (H_{struct} exact match 87.5% vs 19%; 20NG ARI 0.430 vs 0.239 at default $K=3$, Newman reaches 0.557 when $K=4$ is forced; same direction at $N=100$), against the naïve theoretical expectation that CPM should win. **Positioning:** LayerForge shares its core substrate (sentence-embedding similarity + community detection) with prior work in clustering-based topic modeling, notably G2T (Zhang et al. 2023). The contribution of this paper is twofold: (i) the implementation report of combining SCA decomposition, HERCULES hierarchical skeleton, the 4 ± 1 cognitive constraint, and a Mode A/B/C operational layer, and (ii) the systematic empirical comparison of Newman vs CPM in the small-N text regime, including a three-failure-mode mechanism analysis (§5). Concurrently with this work, Bohdal et al. (Samsung 2026) independently arrived at the same problem space on the LLM memory token side; we treat this convergence as evidence that the design space is being identified by multiple independent groups. We report an empirically derived method-selection rule (concept-only; runtime implementation deferred because in the studied domain the rule would output "newman" in essentially all cells). The artifact is published as a Claude Code skill (LayerForge itself does not hold an API key; LLM access at Boundary 1/2 is mediated through Claude Code), and is reproducible end-to-end.

§1. Motivation –関心毎に収束したい

自然言語による AI との対話には、一見「便利」だが構造的に 不利益を生む特性がある:

AI は問いに答えるとき、関連する情報を包括的に提示しようとする。だがユーザーが知りたい「関心」はその対話の瞬間において 通常 1 つである (例: 「この PR の CI/CD だけ知りたい」、「実験設計の前提だけ確認したい」)。

包括的回答は次の三つの形でコストを生む:

1. 読み手の認知負荷 人が一度に保持できる意味単位は短期記憶の容量に縛られる。Cowan (2010) は中心的な working memory capacity を 3-5 chunks と推定している。AI 出力がこれを超えて「関心外の話題」も含むと、読み手は 関心に該当する部分の抽出に追加の認知コスト を払う。
2. multi-agent 連携の帯域浪費 AI agent A の出力を AI agent B が下流で処理する場合、A の出力が包括的だと B は A の出力全体から関心領域だけ拾うコストを払うか、または 関心外の情報に引きずられて drift するかのどちらかになる。

3. **AI コストの線形浪費** full context を毎ターン処理することで、token 単価ベースのコストが線形に上昇する。関心毎の本質情報量は full context より小さいことが多いため、本来不要な計算が含まれる。

これら 3 つは独立な現象だが、**AI 出力が「関心毎に収束していない」** ことが 重なる原因 として 3 者すべてに寄与する (token コストは context window 全体の再処理にも依存するため source は完全一致ではないが、関心毎収束が部分的に解になる)。LayerForge は出力の収束を介して 3 者すべてに同時に影響を与える。

1.1 Research question

AI の出力を 関心毎に対応する layer に収束させる ことで、上記三つのコストを同時に削減できるか？ また、その「layer 化」を AI を再投入せずに 決定論的に 行えるか？

第二の問いが重要である。AI 出力を AI で絞ると、同じ入力に対し異なる出力が得られる (LLM の確率的挙動) ため、関心毎収束の再現性が失われる。本研究は 入力境界での AI 介入のみを許し、絞り込み層自体は決定論的に実装する。

なお本稿で「決定論」と呼ぶのは 解析コア (R3 で詳述) の性質 であり、入力境界 (Boundary 1: 自然言語 → 構造化)・出力境界 (Boundary 2: 構造化 → 自然言語) で optional に介入する LLM は本性質に含まれない。LLM 介入を含めた end-to-end の決定論性は、Boundary を mechanical fallback に切り替えた場合に成立する (§3.1 参照)。

1.2 Related work and positioning

LayerForge の design space (sentence embedding + community detection on similarity graph + per-cluster representation + clustering-driven context compression) は周辺研究と重なる部分が多く、prior art に対する position 整理が必要である。以下、隣接する 4 群の主要文献を整理し、最後に LayerForge の差別化点を明示する。

1.2.1 Clustering-based topic modeling and graph-based text clustering 文書/passage embedding を clustering し topic / cluster 構造を取り出す系列:

- **BERTopic** (Grootendorst 2022, arXiv:2203.05794) -transformer-based document embedding + UMAP 次元圧縮 + HDBSCAN clustering + class-based TF-IDF。Per-document **single topic** assignment が default。
- **Top2Vec** (Angelov 2020, arXiv:2008.09470) -Doc2Vec / Sentence Transformer joint embedding + UMAP + HDBSCAN。topic 数は自動決定。
- **CETopic** (Zhang, Fang, Chen, Namazi-Rad, NAACL 2022, "Is Neural Topic Modelling Better than Clustering?") -SimCSE embedding + UMAP + K-means + TF-IDF×IDFi variant。Neural topic model 系列との直接比較を実施。
- **Vec2GC** (Rao & Chakraborty 2021, arXiv:2104.09439) -term/document の weighted similarity graph 上の community detection、density-based + hierarchical clustering をサポート。
- **G2T (Graph2Topic)** (Zhang, Liu, Yan 2023, arXiv:2304.06653) -pretrained LM embedding → semantic graph 構築 → community detection で topic identification → TFIDF variant で word-topic distribution。本研究の **design substrate** に最も近い **prior art**。Automatic evaluation で BERTopic / CETopic に対し SOTA を主張。
- **SCA** (Eichin et al. 2024, arXiv:2410.21054) -clustering の上で **per-document multi-topic distribution** を導入、単純 clustering の単一所属仮定を緩和。本研究の R3 (distillation) の元実装。
- **LLM-Assisted Topic Reduction for BERTopic** (Janssens, Bogaert, Van den Poel 2025, arXiv:2509.19365) -BERTopic の出力に LLM を merge step として追加する hybrid 系列。本研究の決定論性 (R5) と方向が異なる。

これらは「document/passage を topic 群に分類する」task formulation を取る。BERTopic / Top2Vec / CETopic は density-based (HDBSCAN) または K-means が主流で community detection は使わず、Vec2GC / G2T が graph-based community detection 系列。G2T は本研究と最も近い **substrate** だが、community detection algorithm 名 (Louvain / CPM / Leiden) が abstract レベルで明示されておらず、Newman vs CPM の systematic 比較は内部で行われていない (本研究の §4-§5 の独自性が残る差)。

1.2.2 Hierarchical RAG and GraphRAG 階層構造 + retrieval / summarization の組合せ系列:

- **RAPTOR** (Sarathi et al. 2024, ICLR) -community detection + LLM summarization で hierarchical RAG。
- **GraphRAG** (Edge et al. 2024, arXiv:2404.16130) -Leiden + LLM summarization で knowledge graph 構造化、query-focused summarization。
- **HippoRAG** -PageRank-based retrieval、graph 構造を index 側に使う。
- **Core-based Hierarchies for Efficient GraphRAG** (Hossain & Sariyüce 2026, arXiv:2603.05207) -GraphRAG の Leiden clustering を **k-core decomposition** に置換し deterministic な hierarchy を得る。Leiden 批判の根拠が”modularity optimization admits exponentially many near-optimal partitions on sparse graphs”で、本研究が §5 で引用する Good (2010) Q degeneracy と同じ問題を別 **angle** (置換) で攻める関係。LayerForge は modularity family 内に留まり failure mode を documents する方向。
- **RAG vs. GraphRAG: A Systematic Evaluation** (Han et al. 2026, arXiv:2502.11371) -RAG / RAPTOR / HippoRAG2 / GraphRAG の統一評価フレームワーク提案。LayerForge は本 framework の「passage graph + decoupled summarization」象限に位置する。
- **GraphRAG-Bench** (Xiao et al. 2025, arXiv:2506.02404) -GraphRAG family の domain-specific reasoning benchmark。
- **HERCULES** (bandeerun/pyhercules, MIT 2025) -LLM-integrated hierarchical KMeans の reference implementation。peer-reviewed publication は本研究時点で未刊行、本稿は引用 base ではなく設計 skeleton として参照する位置付け (§2.4)。

これらは大半が解析 pipeline に LLM を組み込む前提を持つ。LayerForge は決定論性 (R5) を理由に解析コアから LLM を排除、LLM 介在を Boundary 1/2 (optional) に限定する点が architectural な差。Core-based GraphRAG とは”Leiden の reproducibility 問題への対応”という problem 認識を共有しつつ、置換 (k-core) ではなく failure mode の documentation + method-selection signal という解決方向で分岐する。

1.2.3 Context and memory compression LLM の context budget 制約下で input を圧縮する系列 (近年の直近系研究):

- **Clustering-driven Memory Compression for On-device LLMs** (Bohdal, Saha, Michieli, Ozay, Ceritli 2026, arXiv:2601.17443) -LayerForge と framing が最も近い同時期 prior art。 ”groups memories by similarity and merges them within clusters prior to concatenation” という mechanism は本研究の Mode C (multi-agent 帯域削減) と同型。ただし対象は **user-specific memory tokens** (personalization 用) であり、目的は context budget 制約下の personalization 維持。LayerForge とは「対象 (memory vs corpus passage)、目的 (personalization vs 関心毎収束)、cognitive constraint (なし vs Cowan 4±1 明示)、method (similarity averaging-then-merge vs community detection on similarity graph)」の 4 点で differentiate される (§1.2.5 で詳述)。
- **EDU-based Context Compressor** (Zhou et al. 2025, arXiv:2512.14244) -Elementary Discourse Unit decomposition による structure-then-select 圧縮。compression mechanism が clustering ではなく discourse 構造解析という点で本研究と直交。

本系列は本研究と「clustering-driven context compression」という framing を共有する。本研究の §1 の動機 (認知負荷 / 帯域 / コスト 3 軸) と Bohdal et al. の ”balances context efficiency and personalization quality” が独立に同じ problem space に到達している事実は、本研究が単独の design ではなく 2025-2026 年に複数研究者が並行して取り組む problem space に位置することを意味する。

1.2.4 Community detection method comparisons 複数の community detection 手法を benchmark で systematic 比較 / 理論分析する系列:

- **Lancichinetti & Fortunato (2009)** -LFR synthetic benchmark を提案、community detection algorithm の標準的評価基盤。
- **Aldecoa & Marín (2013, Scientific Reports)** -Newman modularity / CPM / RB / RN / MCL / Infomap 等を closed benchmark で系統評価、CPM が「benchmark の両端の易しい領域でのみ精度が出る」「ある程度の不安定性を示す」と報告。本研究の §4-§5 の small-N text 結果はこの synthetic 結果との対比軸として位置付け可能。

- Traag et al. (2011, 2019) -CPM の resolution-limit-free 性 (2011)、Leiden refinement (2019) の理論基礎。
- From Leiden to Pleasure Island: The Constant Potts Model as a Hedonic Game (Felipe, Avrachenkov, Menasche 2025, arXiv:2509.03834) -CPM を hedonic game (potential game) として再定式化、local utility 最適化が pseudo-polynomial time で equilibrium に収束することを証明、robust partition criteria を導入。本研究の §5 (CPM の failure mode 分析) と直近の関連だが、彼らは synthetic / random graph ベース、本研究は small-N text similarity graph で empirical に観察を取る相補的關係。

これらは合成 graph / closed benchmark / 理論分析が中心であり、small-N text similarity graph (LayerForge の運用域 $N=20-40$) における Newman vs CPM systematic 比較 は本研究時点では我々が見つけられた範囲では他に存在しない。

1.2.5 LayerForge positioning 上記 prior art に対し本研究の差別化点は以下 5 点:

1. vs G2T (Zhang et al. 2023): 共通 substrate (sentence embedding + community detection on similarity graph) を持つが、本研究は Newman vs CPM の small-N text 領域での systematic 比較 + 3 failure mode 分類 (§5) に focus、加えて SCA distillation step (R3) と HERCULES skeleton (R4)、 4 ± 1 cognitive constraint (R2)、Mode A/B/C application layer を組合せ。G2T との直接 head-to-head 比較は本稿の scope 外 (future work)。
2. vs Clustering-driven Memory Compression (Bohdal et al. 2026, Samsung): clustering-driven context compression という最上位 framing は完全に一致する同時期 prior art。差別化は以下 4 軸-(i) 対象: LayerForge は corpus passage、Bohdal は LLM 内部の user-specific memory token、(ii) 目的: LayerForge は AI output の関心毎収束 (認知負荷 + 帯域 + コスト 3 軸削減)、Bohdal は personalization 維持下の context budget 適合、(iii) method: LayerForge は community detection on similarity graph (Newman / CPM 切替)、Bohdal は similarity averaging + merge、(iv) cognitive constraint: LayerForge は Cowan (2010) 4 ± 1 を ergonomic 上限として明示採用、Bohdal は context budget による外部制約のみ。両研究は 同じ problem space に独立に到達した近接研究 であり、本稿は corpus-passage 側で community-detection 系の手法選択 (Newman vs CPM) を実証評価する complementary な貢献を提供する。
3. vs BERTopic / Top2Vec / CETopic: 本研究は per-passage hard partition を一次成果物とし、layer 構造の上で deterministic Mode A/B/C を提供する。BERTopic 系列は per-document topic distribution を一次成果物とする task formulation の違いから、direct baseline 比較対象には適さない (§2.1)。
4. vs RAPTOR / GraphRAG / Core-based GraphRAG / HERCULES: 本研究は 解析コアから LLM を意図的に decouple し、Boundary 1/2 に限定する (R5)。これは関心毎収束の 再現性保証 を目的とした design choice であり、LLM 統合系列とは正反対の方向。Core-based GraphRAG と "Leiden の reproducibility 問題" の認識を共有しつつ、置換 (k-core) ではなく modularity family 内での failure mode documentation + method-selection signal 構成 という方向で分岐。
5. vs Aldecoa & Marín / Traag / Felipe et al. (Pleasure Island): 既存の community detection 比較は synthetic / random / closed benchmark 中心で、small-N text domain での「resolution-limit-free が機能するか」の empirical 検証は本稿で初めて (我々が確認できた範囲で)。本研究の §4 / §5 の貢献の核がここにある。

つまり LayerForge は G2T の design space に SCA + HERCULES + 4 ± 1 + Mode A/B/C を載せた application であり、core 貢献は (i) その組合せの実装報告、(ii) Newman vs CPM の small-N text 領域での systematic comparison + 3 failure mode 分類 (§5) の 2 点に絞られる。「community detection method comparison そのものを新規 contribution と主張する」のではなく、「既知の method 群を本 domain で実証評価して挙動を文書化した」と読まれるべき report である。Bohdal et al. (Samsung 2026) との同時期 framing 重複は本研究の novelty を否定するものではなく、両研究が独立に同じ problem space を identify した事実 として記録する。

§2. Why these technologies -5 つの技術選択の必然性

§1 の動機から要件を取り出すと:

要件	技術選択	関連文献
(R1) 「関心毎の単位」を corpus から抽出する	Community detection on similarity graph	Newman (2006), F
(R2) 読み手の認知に易しい上限	4±1 layer (Cowan の working memory capacity)	Cowan (2010), Mil
(R3) 各 layer の” 中核情報” を取り出す	Per-layer distillation (SCA-style)	Eichin et al. (2024)
(R4) 大規模 corpus への拡張性	Hierarchical KMeans foundation (HERCULES-style)	Reference impl bar
(R5) 「AI 出力を AI で絞らない」決定論性	LLM-free analysis core, AI 介在は境界のみ	LayerForge design

2.1 (R1) なぜ community detection か

「関連する文の集合」を教師なしで自動抽出する手法として確立済み。RAG 拡張 (RAPTOR, GraphRAG, HippoRAG) も同類の発想で passage clustering に Louvain/Leiden を使う。本研究はその一族だが、後述 (R5) の制約から community detection を LLM summary と decouple している点異なる。

なお、document-level の topic 分布を出す手法 (LDA, BERTopic 等の topic modeling 系列) は passage-level の hard partition を直接出すのが本来の目的でないため、本研究の「各 passage がどの layer に属するか」という task formulation と直接比較する baseline には適さない。本稿の比較対象は community detection method 内部の選択 (Newman vs CPM) に限定する。

2.2 (R2) なぜ 4±1 か

Cowan (2010) は短期記憶の中心的容量を 3-5 chunks と推定。「関心毎収束した結果が読み手に易しい」と要求するなら、layer 数の ergonomic な上限をここに置くのが自然。これは「4±1 が universal な認知定数」と主張するのではなく、設計の目標値として採用するというデザイン判断。

2.3 (R3) なぜ SCA distillation か

community detection は「どの文がどの layer に属するか」を出すのが、layer の代表表現 まで出さない。読み手 / 下流 agent が layer に渡すには「この layer は何の話題か」が短く分かる形が必要。Eichin et al. (2024) の Semantic Component Analysis は per-cluster の代表 component を distill する手法で、本要件と整合する。

2.4 (R4) なぜ HERCULES か

flat 1-level community detection で本研究の主要結果は十分得られるが、将来的に「N=10K の large corpus を $4 \times 4 \times 4 \times 4 = 256$ のような階層に再帰展開する」運用を想定し、hierarchical KMeans の skeleton を予め採用 (現実装は max_depth=1 default、option で recursive 化可能)。

なお HERCULES は本研究時点では GitHub の reference implementation (bandeerun/pyhercules、MIT) のみ存在し peer-reviewed publication なし。本研究は HERCULES を「hierarchical KMeans を採用した先行 OSS implementation」として position 参照しており、HERCULES 自体の理論的優位性を引用 base にしているわけではない。R4 の本質的根拠は「再帰的 KMeans が large corpus 拡張に skeleton として機能する」という設計判断であり、HERCULES はその一実装例 (= 参照先) という位置付け。

2.5 (R5) なぜ決定論コアか

§1 で述べた通り、LLM の確率的挙動は関心毎収束の再現性を破壊する。同じ入力で異なる layer 化が起きたら「収束後の出力」を信用できない。本研究は:

- 入力境界: 自然言語 → 構造化 (Boundary 1) で LLM 介在可
- 解析コア: 決定論 (community detection + KMeans + SCA は全て決定論アルゴリズム)
- 出力境界: 構造化 → 自然言語 (Boundary 2) で LLM 介在可 (optional、本研究では Mode A/C は AI 不要で動作)

この制約により、解析結果は 同じ input + 同じ hyperparameters → 同じ output を保証する。ここで hyperparameters には community-detection method の選択 (community_method="newman" | "cpm")、random seed、target_range、embedding model 等を含む。これらを固定した上での再現性であり、method を切り替えれば異なる partition が得られうる (§4.3 で実証)。

2.6 まとめ-組合せの必然性

5 つの技術選択はそれぞれ独立の要件 (R1-R5) から導かれており、逆に言えば各要件を満たす別の選択は本研究の課題に合わない:

- community detection を skip → 関心単位の自動抽出ができない
- 4±1 を捨てる → 認知負荷削減の根拠を失う
- SCA を skip → 各 layer の代表表現が出ない
- HERCULES を捨てる → 大規模拡張の base がない
- 決定論を捨てる → §1 の問題を AI で AI を絞ろうとして自己矛盾

この組合せが §1 の動機 (R1-R5) を実装上充足する 設計根拠を §3 で示す。組合せ内部の 最重要選択 (community detection method の Newman vs CPM) の妥当性を §4 で empirical に検証する。SCA distillation / HERCULES hierarchy / 4±1 制約 / 決定論性の個別 ablation は本稿の scope 外であり、§7.5 で明示する。本稿の主張は「組合せ内部の最重要選択について empirical に method 比較を行い、本 domain で Newman backend が適合することを示した」に限定される。

§3. Approach -LayerForge 実装

3.1 Pipeline

[input: list of passages, optional query]

(B1) parse_to_structure - 自然言語 → FormulationInput (optional LLM, has mechanical fallback)

build_similarity_matrix - sentence-transformers cosine similarity

find_valid_scale / find_cpm_resolution
 - threshold (Newman) or (CPM) で 4±1 範囲の K を探す

detect_communities - Newman spectral + KMeans / CPM-Louvain (community_method option)

compute_modularity - Q (両 method) + cpm_h (CPM 時)

distill_layer - per-layer SCA (UMAP + HDBSCAN-style components)

hierarchy_to_layer_summaries - { layers: [...], inter_layer_relations: [...] }

(B2) render_to_natural - 構造化 → 自然言語 (optional LLM, optional)

[output: 4±1 layers + per-layer reps + optional natural-language rendering]

3.2 Three modes of operation

Mode	CLI	目的
A (decompose)	layerforge-decompose	corpus を 4 ± 1 layer に分解、UI から「興味のある layer のみ」を引ける
B (decide)	layerforge-decide	意思決定情報を layer 化、open/close/settle で関心の遷移を追跡
C (compress)	layerforge-compress	AI の verbose 出力を query に関連する layer subset に圧縮 (decision-less sub)

3.3 Claude Code skill form

LayerForge は Python CLI として独立動作するが、運用上は Claude Code skill として配置する設計を採用した:

- LayerForge 本体は API key を保持しない (Boundary 1/2 で LLM 介在を行う場合は Claude Code 経由で実行、API key は Claude Code 側で管理)
- PostToolUse hook で出力 schema を自動検証
- `.claude/skills/layerforge/SKILL.md` が 使う側の Claude への指示書 として機能

この形態により、LayerForge を「LLM の周辺ツール」として既存 AI workflow に挿入可能で、解析コアは LLM 不在の決定論を保つ (Boundary 1/2 は §3.1 の通り optional に LLM 介在可、mechanical fallback あり)。

3.4 Dual community-detection backend

本研究の中心的な engineering 選択は community detection method。実装は両方を提供:

- Newman (default): threshold + spectral algorithm + KMeans on embeddings、modularity Q で品質測定
- CPM (opt-in): Leiden-CPM (self-implemented, MIT-pure)、(n_c choose 2) quadratic penalty 付き H 関数を最大化

切替は community_method option で行う:

```
layerforge_core(input, community_method="newman") # default
layerforge_core(input, community_method="cpm")    # opt-in
```

3.5 Reproducibility infrastructure

アーティファクト

```
scripts/k_sweep/correlation_data.py (119 rows)
scripts/k_sweep/heatmap_N_x_K.py (196 attempted cells =  $7 N \times 14 K \times 2$  methods, of which 167 succeeded -残り 29 は)
scripts/k_sweep/cpm_compare.py (95 rows + ARI/NMI)
scripts/k_sweep/multi_corpus_verify_v2.py (8 conditions)
scripts/k_sweep/k10_multi_corpus.py
scripts/k_sweep/run_robustness.py (32 setting-method aggregates, each holding 8 K-range measurements; 256 K-range)
tests/integration/test_real_data_20ng.py
tests/axioms/test_cpm_karate_club.py
```

全 raw CSV / JSON / PNG が git に同梱。コミット hash で固定。

§4. Empirical findings –結果として成立した観察

本節は「結果として成立することが分かった」報告として記述する。「優れている」「最良である」といった主張は避け、測定された数値とその範囲を中心に据える。

4.0 K に関する用語整理 (本節の前提)

用語	定義	使用箇所
target_range	探索したい K の range (例: $4 \pm 1 = (3, 5)$)	§3.1 find_valid_scale / find_cpm_resolution の
K_actual	実行結果として得られた community 数	全 figures / tables
K_optimal	K sweep の中で Q が最大になる K (Newman)	§4.1, §4.4
Q_peak K	K_optimal と同義	§4.1
n_themes	corpus が持つ ground truth な theme 数	§4.4 (H_struct), §4.6 (20NG)
K=10	特定の運用候補値 (AI input compression)	§4.5
default K	method-selection rule での probe 値 (本稿では K=4)	§5.7

§4.1 は target_range を 4 ± 1 に固定せず広めに sweep し Q peak を測定する。§4.4 では各 setting で n_themes に対応する K_actual の tracking を測定する。両者の corpus 設定の差は §4.4 で明示。

4.1 Q peak K の N-dependent 挙動 (Newman)

本 sub-section の corpus は cross-domain mpnet corpus (KDF-perovskite project の real-world markdown corpus、philosophy / exploration / proof / blog の 4 themes、n_themes = 4) を per_theme = 2, 3, 4, 5, 6, 8, 10 で sampling し N を変動させたもの (§4.4 の synthetic disjoint-vocabulary corpus とは異なる corpus family、両者の関係は §4.4 末尾 reconciliation で詳述)。「同一 corpus 構造に対し sampling 密度を変えると Q peak K が動くか」を測る (Good 2010 の theoretical degeneracy が本実装で再現するかの check)。

7 N values $\times$ 14 K values $\times$ Newman backend での Q max を取った結果:

N	Q peak K	Q peak value
8	3	0.403
12	6	0.778
16	11	0.720
20	6	0.716
24	9	0.646
32	4	0.550
40	6	0.610

観察: 同一 corpus 構造に対し N (sample size) を変えると Q peak K が 3~11 の範囲で揺れる。これは Good et al. (2010) が理論的に確立した Newman Q の degeneracy が本実装でも実証されたことを示す。本研究の独自貢献はこの発見自体ではなく、N $\times$ K 軸上での実証データセットである。

4.2 Above-limit fraction の method-agnostic monotone 性

Fortunato & Barthélemy (2007) の resolution limit ($\sqrt{L/2}$) を基準とし、各 community が limit を超える割合を計算。本節では method-agnostic 主張を直接支持する common-reference 測定 (scripts/k_sweep/data_current/cpm_compare_data.csv、両 method を corpus 全体の median similarity で threshold した同一 graph 上で比較) を主結果として提示する:

N	K	Newman above-limit	CPM above-limit
12	3	0.67	0.67
12	4	0.25	0.25
20	4	0.50	0.50
24	4	0.75	0.75
24	5	0.60	0.60

N	K	Newman above-limit	CPM above-limit
40	5	0.80	0.60

観察: 6 cells のうち 5 cells で両 method の above-limit fraction が完全一致、唯一 N=40, K=5 で divergent (Newman 0.80 / CPM 0.60)。Newman も CPM も K の増加に対し monotone-decreasing で、両者の曲線は本研究の運用域 (N=12-40) でほぼ同形。N=40 の divergence は §4.3 が示す「N 32 で partition が divergent になる regime」と整合する (同一 graph でも method 間で partition 形状が異なれば cluster size 分布が異なり、above-limit fraction も離れうる)。

→ above-limit fraction が community detection method に依存しない supplementary K-selection signal として機能することが本研究で実証された (本検証範囲)。なお、Newman 自身の find_valid_scale が選んだ で測った asymmetric reference 版の同 cell 数値は §5.2 に再掲し、機序分析側で扱う (両者の reference 差から得られる洞察を分離するため)。

4.3 Newman vs CPM partition agreement (ARI by N)

両 method を同じ (N, K) cell で実行し、partition 間の Adjusted Rand Index を測定:

N	ARI mean	ARI max
12	0.871	1.000
20	0.728	1.000
24	0.689	1.000
32	0.490	0.779
40	0.411	0.842

観察: N 24 では partition がほぼ一致 (一部 K で完全一致)、N 32 では divergent (最大一致でも 0.85)。「Newman でも CPM でも同じ結果」と素朴に期待するのは本 domain の小 N 側でのみ妥当で、運用域 (N=20-40) では method choice が partition 構造に影響することが分かった。

4.4 H_struct (Q peak K = n_themes) tracking

本 sub-section は §4.1 とは異なる軸 を測る:

- §4.1: 「同一 corpus 構造で N が変動した時 Q peak K がどう動くか」(Q degeneracy の reproduction)
- §4.4: 「異なる n_themes corpus でそれぞれ Q peak K が n_themes に一致するか」(H_struct hypothesis の test)

16 settings = 4 n_themes (3, 4, 5, 7) × 2 embedders (MiniLM, mpnet) × 2 seeds (42, 123)。各 setting で K range を {1-2, 2-3, 3-4, 3-5, 5-7, 6-8, 8-10, 10-12} に sweep。range の構成意図は二段: (a) 全 K=1 から 12 を重複を許して連続 cover することで find_valid_scale の binary search 安定性を K 域全体で確認、(b) Cowan 4±1 (=3-5) 周辺は 2-3, 3-4, 3-5 の 3 ranges で密に cover することで本研究の関心域 (4±1) での誤差を小さくする。各 range で Q が最大となる K_actual を取得し、対応する n_themes と比較:

- Newman: 14/16 (87.5%) 完全一致 (K_actual == n_themes)
- CPM: 3/16 (19%) 完全一致

観察: 「Q peak K が corpus の自然な theme 数を tracking する」という仮説 (H_struct) は本 16 settings (synthetic disjoint-vocabulary corpus) の domain で Newman backend に強く支持され、CPM backend では支持されなかった。なお 16 settings は small sample のため、binomial test 等の formal な統計的有意性検定は本稿では実施せず、効果サイズ (87.5% vs 19%) の大きさを定性的に評価する。

Scope 限定 (重要): 上記 87.5% tracking は synthetic disjoint-vocabulary corpus 上での結果であり、「Newman の find_valid_scale pipeline が任意 corpus で n_themes を自動 tracking する」という強い主張は本稿では行わない。External benchmark (§4.6 で示す 20 Newsgroups, N=100) では default

の `target_range=(3,5)` 設定で Newman ですら $K=3$ を返し $n_themes=4$ を tracking 失敗する。これは disjoint-vocabulary という synthetic 条件と公開 corpus の vocabulary overlap の差を反映したもので、§5.4 (find_cpm_resolution 同型の calibration 設計が target_range 下端 K を選好する点) で機序的に位置付ける。本節の `H_struct` 主張は synthetic corpus 条件下での部分支持 と読まれるべきである。

→ **Figure:** figures/new/fig_h_struct.png -2-panel scatter (Newman | CPM) で K_actual vs n_themes 、対角線が perfect tracking を示す。Newman side は対角線周辺に集中、CPM side は systematically 下方に外れる (under-merging を視覚化)。

§4.1 との関係 (reconciliation): §4.1 と §4.4 は異なる corpus family を用いる:

項目	§4.1
Corpus 起源	real-world cross-domain (KDF-perovskite project の philosophy / exploration / proof / blog markdown)
軸変動	同一 corpus 構造で sampling 密度 (per_theme) を変動
主測定	N に対する Q peak K の揺れ
結果	Q peak K が 3-11 で N -依存に揺れる (Q degeneracy reproduction)

両者は異なる corpus family の異なる質問 を測っており、矛盾しない: §4.1 は real-world corpus で「同一 corpus 内の sampling 密度変動による degenerate effect」、§4.4 は synthetic corpus で「異なる n_themes を持つ corpus 間の tracking」を測る。Real-world cross-domain corpus が §4.4 の synthetic corpus より vocabulary overlap が大きいため、§4.1 で Q peak K が $n_themes=4$ に固定しないのは想定内の挙動である。

4.5 $K=10$ self-routing (AI input compression candidate)

self-routing accuracy の定義: 各 passage p について、 p のテキスト自体を query とみなし、 p の embedding を全 layer centroid と比較、最も近い centroid の layer を選ぶ。chosen layer が p の所属 layer と一致した割合。これは LLM-side の paraphrased query routing よりも weak な test であるが、algorithm-level の partition consistency 確認には十分。production query routing の確認は本稿の scope 外 (§7 Limitations 参照)。

8 conditions (2 corpora $\times$ 2 embedders $\times$ 2 methods) $\times$ $K=10$ cell:

condition	Newman self-routing	CPM self-routing
same-domain 5themes / MiniLM	30/30 (100%)	29/30 (97%)
cross-domain 4themes / MiniLM	24/24 (100%)	23/24 (96%)
same-domain 5themes / mpnet	30/30 (100%)	30/30 (100%)
cross-domain 4themes / mpnet	24/24 (100%)	24/24 (100%)

観察: $K=10$ で self-routing が 96-100% に留まり、partition consistency 指標上の理論最小読込量比は $1/K = 10\%$ (10x reduction)。両 method で類似の精度。self-routing 指標上は AI 入力圧縮としての $K=10$ が method-agnostic に成立する。

重要な scope 制限: 上記の「10x reduction」は partition consistency の理論最小値 であり、production で paraphrased query を投げた時の retrieval 精度は本節では未測定。self-routing が測るのは「passage 自身を query にして所属 layer に戻るか」という intra-cluster tightness で、production query routing (パラフレーズや query-passage semantic gap を伴う) の精度との間には gap が存在する。本数値を「AI 入力 10x 圧縮の達成」と読み替えるのは飛躍であり、本節は partition consistency 上の必要条件 が満たされたことの報告に留まる。production query routing の精度確認は本稿 scope 外 (§7.2 / §7.5 参照)。

§4.4 / §4.6 との粒度依存性: 本節 ($K=10$) と §4.4 ($K = n_themes$) で method 差の大きさが大きく異なる現象は、§5 で機序的に説明する。要旨: $K = n_themes$ 域では Newman が真の thematic structure を Q peak で拾うのに対し CPM は under-merge するため method 差が顕在化する。一方 $K=10$ のような over-clustering 域では両 method とも「more clusters than themes」へ強制されるため、ground truth とは別に partition の細分化が支配し method 差は出にくくなる。粒度域の違いが method 差の顕在化を決める。

4.6 20 Newsgroups (external benchmark)

4 thematically distinct newsgroups (sci.med / sci.space / rec.sport.hockey / talk.politics.guns)、25 docs/topic = **N = 100 documents** (本 sub-section のみ、§4.1-4.5 の N=20-40 から scaling check を兼ねる)。

測定 **K**: 両 method とも `target_range = (3, 5)` (Cowan 4 ± 1 の default range) で実行。本実装の現行 default 動作では両 method とも **K=3** を返す (`tests/integration/test_real_data_20ng.py` の許容 **K** 集合: Newman {3,4,5}、CPM {2,3,4,5})。両 method とも **K=3** で頭打ちになる現象は、本実装の **K calibration** 設計 (`find_valid_scale` / `find_cpm_resolution` がいずれも `target_range` 下端の **K** を選好する仕様、§5.4 で機序解説) の symptom であり、`n_themes=4` の正解にたどり着けないのは Newman の本来的限界ではない。K=4 強制時の Newman ARI=0.557 が calibration を経由しないときの本来の到達点を示す。得られた partition を **ground truth** (`n_themes = 4`) と比較、Adjusted Rand Index (`sklearn.metrics.adjusted_rand_score`) で測定:

method	K_actual (default)	ARI vs ground truth
Newman	3	0.430
CPM	3	0.239
chance baseline (random partition)	–	0

参考として、**K** を 4 に固定した場合の Newman ARI は **0.557**、**K=5** で 0.313。CPM は **K=3,4,5** すべてで ARI 0.24 と一定 (under-merging により **K** を変えても同じ partition family に落ちる)。

→ **Figure**: `figures/new/fig_20ng_ari.png` -bar chart (Newman / CPM × **K=3,4,5**)、chance baseline 線と Newman **K=4** best (0.557) を annotation で強調。default **K=3** で Newman 1.8x、**K=4** 強制で Newman 2.3x の差が視覚的に明確。

観察: 公開 benchmark での external validity は default 設定で **Newman backend** が **CPM backend** を約 1.8 倍上回る (Newman 0.430 / CPM 0.239)。両者とも chance baseline は明確に超えるが、Newman の優位は安定して観察される。**K=4** を強制した場合の Newman ARI=0.557 が示すように、Newman 側は適切な **K** で更に高い ARI に到達可能。CPM 側は **K** を変えても 0.24 で頭打ちとなり、§5.3 の (`n_c choose 2`) penalty effect (mode (c)) と整合する。

§5. Why CPM Underperforms –三つの failure mode と運用方針

§4 で観察された結果のうち最も予想と異なったのは CPM が Newman より一貫して劣る ことである。Traag et al. (2011) は CPM が resolution-limit-free であることを subgraph-invariance 性質で数学的に示しており、Newman の Q degeneracy (Good 2010) に対する 正しい修正 とされる。にもかかわらず本研究の small-N text domain では CPM が逆方向に劣化した。本節は 三つの異なる failure mode に切り分けて機序を整理する。Karate Club 上の **K=4** over-split (§7.3) と 20 Newsgroups 上の **K=3** under-merge (§4.6) という反対方向の挙動も、別 mode として一貫に位置付けられる。

5.1 三つの failure mode

CPM-Louvain が ground truth から外れる原因は本研究の検証範囲で 少なくとも三つの独立な機序 に分離できる:

Mode	内容
(a) Louvain refinement gap	vanilla Louvain の single-node move では到達不能な macro partition がある (Traag
(b) Calibration bias	本実装の <code>find_cpm_resolution</code> は を bisection し、 <code>target_range</code> 内で smallest I
(c) (<code>n_c choose 2</code>) penalty effect	small-N で resolution limit が binding しない regime では、CPM の quadratic penal

本稿の数値ギャップの帰属:

- §4.4 H_struct 87.5% vs 19% (synthetic disjoint-vocabulary、各 setting で K_actual を Q peak で取得) → 主に (c)。K range が n_themes 周辺を密に cover しているため (b) の影響は小さい
- §4.6 default K=3 (20NG) → (b) calibration bias が dominant。両 method とも target_range=(3,5) の下端 K=3 に収束
- §4.6 K=4 強制下の Newman 0.557 vs CPM 0.24 → (c) penalty effect。calibration を回避しても CPM が劣る成分
- §7.3 Karate K=4 over-split → (a) Louvain refinement gap。 を変えても K=2 macro split に到達できない

5.2 目的関数の構造差

両 method の objective function を比較する:

- Newman: $Q = (1/2L) \sum_{ij} [A_{ij} - (k_i k_j / 2L)] (c_i, c_j)$
 - 内部 edges を null model (期待値 $k_i k_j / 2L$) と比較
 - cluster サイズに対する明示的 penalty なし
- CPM: $H = \sum_c [m_c - \frac{1}{2} \times (n_c \text{ choose } 2)]$
 - intra-edges から クラスタペア数 $\times$ を引く
 - cluster size n_c に対し 二次的に増大 する penalty

→ **Figure:** figures/new/fig_cpm_mechanism.png -cluster size n_c に対する penalty/null contribution の推移を比較プロット。CPM の $(n_c \text{ choose } 2)$ quadratic curve が Newman null-model の線形 reference を超える”under-merging zone”を網掛けで明示。

§4.2 で示した common-reference (両 method を corpus median similarity の同一 graph 上で比較) では、 $N=12-40$ の運用域で above-limit fraction の差が 6 cells 中 5 cells で完全一致する。すなわち本 domain では Newman の resolution limit は operationally bind しておらず、CPM 採用の理論的動機 (limit-free 性) が解こうとしている問題が そもそも本 domain では問題になっていない。残るのは CPM 固有の penalty 構造の作用のみ、というのが §5.3 以降の出発点となる観察である。

参考として、§4.2 と対をなす asymmetric reference (Newman は自身の find_valid_scale が選んだ threshold、CPM は median similarity で threshold) の同 cells を以下に再掲する。Newman の resolution limit が”binding か”を直接見たい場合の reference として有用だが、method 比較の地としては §4.2 の common-reference を採る:

N	K	Newman above-limit (asym)	CPM above-limit (asym)
12	3	0.67	0.67
12	4	0.25	0.25
20	4	0.75	0.50
24	4	0.75	0.75
24	5	0.40	0.60
40	5	0.60	0.60

(Newman 側の差は 選択経路の違いで、graph 上の partition 構造の差ではない。Source: scripts/k_sweep/data_current/head)

5.3 Failure mode (c) $-(n_c \text{ choose } 2)$ penalty effect (small-N text domain)

LayerForge の運用域 ($N=20-40$ cross-domain text corpus) では:

- edge density は中程度 (sparse でも dense でもない)
- §4.2 の common-reference data が示す通り、above-limit fraction が 0.25-0.80 の範囲で推移し $N=20$ ではほとんど 0.5 以上 → Newman の resolution limit は operationally bind していない
- CPM を適用すると $(n_c \text{ choose } 2)$ 項が m_c を相対的に圧倒、「大きい cluster は H 損」が dominant force となり、CPM は systematically coarse な partition を選ぶ (K-matched 比較でも残る under-merging)

これが §4.4 (H_struct 87.5% vs 19%) と §4.6 K=4 強制時の Newman 0.557 vs CPM 0.239 の数値ギャップの根本理由である。Mode (c) は K を強制した状態でも残る成分であり、本 domain における Newman 優位の 構造的 主因。

5.4 Failure mode (b) -find_cpm_resolution の calibration bias

本実装の find_cpm_resolution (および対応する Newman 側の find_valid_scale) は target_range 内で smallest K を preferred する設計である。具体的に cpm_backend.py::find_cpm_resolution (L262-263) は:

```
if K_min <= k <= K_max:
    best = (mid, labels, h, k)
    hi = mid # prefer smaller (smaller K)
```

これは「より coarse な partition は (4 ± 1 制約下で) 解釈しやすい」という運用上の判断 (working memory 上限が ergonomic 上限なら K は小さい方が望ましい) を反映した設計だが、副作用として n_themes が target_range[0] と一致しない corpus では tracking 失敗が生じる。

§4.6 (20 Newsgroups、n_themes=4、target_range=(3,5)) で両 method とも K=3 に収束する現象はこの mode の symptom。K=4 を強制すると Newman は ARI=0.557 まで上昇するが (mode (b) を回避できれば mode (c) の効果のみ残る)、default 動作だけを見ると Newman もまた n_themes を tracking できない。

§4.4 (synthetic corpus、K range が n_themes 周辺を密に cover) で Newman が 87.5% tracking できるのは、K range 設計が n_themes を target_range[0] 近傍に置いており mode (b) の影響が小さいため。External corpus に対しては default の target_range=(3,5) では mode (b) が顕在化することに本稿は注意を促す。

5.5 Failure mode (a) -Louvain refinement gap (cross-ref to §7.3)

本実装の CPM-Louvain は vanilla Louvain (greedy single-node move) であり、Leiden refinement (Traag 2019) は含まない。Karate Club のような dense small graph では K=2 macro split が single-node move の局所最適でなく、を低くしても K=4 sub-clustering に stuck する (文献既知)。本稿の §7.3 で Karate Club の結果 (ARI 0.595, K=4) はこの mode に該当する旨を明示する。

本実装の小規模 text domain での主要結果 (§4.4, §4.6) は dense small graph でないため mode (a) は dominant でないが、外部対比 (Karate Club) で K が ground truth より大きく出る現象はこの mode で説明される。CPM の penalty 構造そのものとは独立の、algorithm-engineering 上の制約である。

5.6 含意-“resolution-limit-free is universally better” は domain 依存

CPM の理論的優位は resolution limit が binding な domain で発揮される。LayerForge の small-N text domain では §4.2 common-reference data が示す通り limit binding がそもそも稀で、CPM の”修正”が解こうとしている問題が本 domain では問題になっていない。同時に、CPM 固有の penalty 構造 (mode (c)) と calibration 設計 (mode (b)) が別の問題を生み、結果として Newman が一貫して上回る。

これは Traag et al. (2011) の理論結果 (CPM の subgraph-invariance) への直接反論ではない。operating domain の特性 (limit binding の有無、典型的な n_themes と target_range の関係、graph の dense/sparse 度合い) と照らした上で method 選択が必要であるという mechanism observation である。

5.7 Empirical method-selection rule (sketch)

§5.2-§5.5 の機序を踏まえると、運用時に「Newman / CPM のどちらを使うか」を決める empirical signal が構成可能である:

Signal	推奨閾値
Newman Q at default K (K=4)	Q 0.3 → Newman

Signal	推奨閾値
Above-limit fraction at default K	0.5 → Newman safe
Edge density at	> 0.5 → consider CPM
Corpus size N	> 1000 → consider CPM
Cross-method ARI at probe K	< 0.3 → flag uncertainty

decision rule の擬似コードは companion document の method-selection rule 節に詳細記載 (GitHub repository: https://github.com/ChaiCroquis/LayerForge/blob/main/docs/08_empirical_findings.md)。code として実装はしていない: LayerForge の運用域 (N=20-40, edge density 中程度) では rule が ~100% "newman" を出力するため、実装コストが情報量を上回る判断による。別 domain (N=1000+, dense graph) への拡張時に foundation として使う設計。

§6. Application examples

LayerForge の Mode A/B/C は §1 の三つのコスト (認知負荷 / 帯域 / AI コスト) にそれぞれ対応する応用域を持つ:

6.1 Mode A –読み手の認知負荷削減

corpus を 4 ± 1 layer に分解、UI / CLI で「興味のある layer のみ」展開可能。reader は full corpus を読まずに、関心 layer の代表表現から内容を判断できる。

6.2 Mode B –意思決定情報の関心遷移管理

考慮中の意思決定情報を layer 化し、open/close/settle 状態を持たせる。決定中の事項 (open) と決定済み事項 (settled) を分離することで、読み手は 今活性化している関心 だけに focus できる。

6.3 Mode C –multi-agent 帯域削減

AI agent A の verbose 出力 (例: 15K token の包括回答) を agent B の query に関連する layer subset に圧縮 (decision-less subset 保証、つまり情報の subset としてのみ機能、要約は LLM が行うわけでない)。本研究の scripts/multiagent_demo/ で 63% の context reduction を達成 (B 軸 null result: 圧縮後でも下流 agent の出力品質は維持された)。

§4.5 の $1/K=10\%$ との関係 (異なる measurement): §4.5 が示した「 $K=10$ で $1/K=10\%$ (10x reduction)」は partition consistency 上の理論最小読込量比 (passage を query にした self-routing が成功する前提で、所属 layer 1 つだけ読めば十分という上限) であり、measurement の単位は「読み込む layer 数 / 全 layer 数」。本節 Mode C の 63% reduction は token 数ベースの実測値で、demo script で agent A の full output と agent B が実際に読み込む subset とを token 単位で比較したもの。両者は異なる measurement (理論最小 vs production 実測) で直接比較できない。前者は $K=10$ の細粒度 partition での floor、後者は Mode C の現実的な workflow での measured reduction。

§7. Limitations

7.1 検証範囲の限定

- 運用想定は N=20-40 small-N text domain。External benchmark として 20 Newsgroups で N=100 まで scaling check を実施 (§4.6)、結論は同方向 (Newman 優位)。§4.3 の ARI 表は N=12-40、§4.6 は N=100 で、中間 N (40-100) 域は本稿で直接 sweep していないが、両端点で Newman 優位が同方向に観察されたことから補間域での挙動も同方向と推測される (確認は future work)。N > 1000 や dense graph は未検証
- External benchmark は 20NG 1 件のみ –他 public corpus での再現は future work

- CPM-Louvain は Leiden refinement なし -文献既知の Karate Club K=4 sub-clustering 現象に該当、§7.3 で具体的に位置付け

7.2 B 軸の null results

LayerForge を AI workflow に組込んだ際の LLM 自体の behavior 改善 は本研究の範囲外で、実測した 3 試行 (hallucination benchmark、multi-agent drift verification、context filter ablation) はすべて **null result** だった。これは「LayerForge が AI を賢くする」claim が defensible でないことを意味する。本研究の貢献は AI 出力を関心毎に絞り込んで読み手 / 下流 agent に渡す pipeline 自体 にあり、AI 行動の改善は claim しない。

7.3 自前実装の correctness gate

CPM-Louvain の reference implementation である GPL `leidenalg` との数値正確性 head-to-head 比較は MIT license との互換性問題で実施せず。代替として以下の 3 段階の間接 gate を設けている:

1. **Synthetic 3-block test** (`tests/axioms/test_cpm_backend.py::test_cpm_partition_separates_three_blocks`): 3 blocks (intra=0.9, inter=0.05) で K=3 を完全 recover
2. **Karate Club ARI = 0.595** vs empirical 2-community ground truth (chance baseline = 0): Zachary (1977) 公開 graph で、Leiden refinement 不在による over-splitting (§7.1 で言及した K=4 sub-clustering 現象) のため partition の cluster 数を直接確認すると K=4 となる。これが ground truth の 2-community split の sub-refinement に該当することは cluster 構造から判定でき、ARI=0.595 は この sub-refinement と ground truth の整合度 を示す中間値である (完全一致なら ARI=1.0、ランダムなら 0)。Leiden refinement を加えれば K=2 macro split に到達する余地があることを示唆
3. **Determinism cross-check**: 同 seed で Newman / CPM とも reproducible

これは「CPM が正しく実装されている」の strong proof ではない (Leiden refinement なしの local optimum 限界が §5.5 / §7.1 で言及済み)。「Newman との empirical 比較を行う前提として最低限の correctness 要件を満たしている」ことの evidence として位置付ける。

7.4 4±1 の universality 主張なし

Cowan (2010) の working memory capacity は本研究の設計目標として採用したが、「4±1 が任意の corpus で universal に emerge する」とは主張しない。実測では corpus に応じて K_{optimal} は 2~11 の範囲で変動 (§4.1)、ergonomic な上限としての位置付けに留める。

7.5 Scope 外として宣言する ablation

本稿は community detection method 選択 (Newman vs CPM) の妥当性検証に scope を限定した。組合せ内部の他の選択肢に関する以下の ablation は実施していない:

- SCA distillation on/off の partition / output quality への effect
- HERCULES hierarchy depth = 1 (本稿の default) vs depth > 1 の effect
- 4±1 制約 vs 制約なし (任意 K_{optimal}) の cognitive load 削減効果差
- Boundary 1/2 で LLM 介在あり/なし の output drift 測定

これらは LayerForge の組合せ全体を本稿より広範に検証する場合に必要なが、本稿の主張範囲 (組合せ内部の最重要選択である community detection method の妥当性) の外にあり、future work とする。

§8. Conclusions

§1 で述べた問題-AI 出力を関心毎に収束させて認知負荷 / 帯域 / コストを削減する-に対し、§2 の 5 技術組合せが task の要件 R1-R5 を充足する設計 であることを §3 で実装、組合せ内部の最重要選択 (community detection method) を §4 で empirical に検証した。

主要な observations (本 domain の本検証範囲での観察):

1. **Above-limit fraction (Fortunato-Barthélemy ratio)** は **community-detection method** に依存しない **supplementary K-selection signal** として機能する (§4.2)。
2. 本検証範囲 (16 settings + 20 Newsgroups N=100) において、**Newman backend** が **CPM backend** を効果サイズの大きく上回った (H_struct 87.5% vs 19%, 20NG ARI 0.430 vs 0.239 at default K=3、Newman は K=4 強制で 0.557 まで到達、§4.4 / §4.6)。小 sample のため formal な統計的有意性検定は実施せず、効果サイズで定性的に報告する。
3. **CPM** の劣化は機序的に説明可能: (n_c choose 2) quadratic penalty が under-merging を引き起こす。"resolution-limit-free is universally better" は domain によらず成立する命題ではない (§5)。
4. **self-routing** 指標 (passage を自身の query にした **partition consistency test**) 上、**K=10** で **1/K=10%** の理論最小読込量比が両 method で成立 (§4.5)。これは production query routing の精度を示すものではなく、partition consistency の必要条件 が満たされたことの報告である。production の paraphrased query routing 精度、および §6.3 で報告した Mode C の 63% 実測 token reduction との関係は §4.5 / §6.3 内に明示、production 精度確認は future work (§7.2 / §7.5)。
5. **Method-selection rule** を **empirical signals** から構成可能 (§5.7、詳細は GitHub repository の companion document: https://github.com/ChaiCroquis/LayerForge/blob/main/docs/08_empirical_findings.md)。本 domain では出力がほぼ常に "newman" のため runtime 実装はせず、docs のみで保存。

LayerForge は MIT license で公開、Claude Code skill として運用可能、再現スクリプトと CSV / JSON を同梱。本稿で測定した数値は publication 時の commit hash で固定される (git log 参照)。

References

Foundational

- Cowan, N. (2010). The magical mystery four: How is working memory capacity limited, and why? *Current Directions in Psychological Science*, 19(1), 51-57.
- Zachary, W. W. (1977). An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4), 452-473.

Community detection theory and benchmarks

- Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS*, 103(23), 8577-8582.
- Fortunato, S., & Barthélemy, M. (2007). Resolution limit in community detection. *PNAS*, 104(1), 36-41.
- Lancichinetti, A., & Fortunato, S. (2009). Benchmarks for testing community detection algorithms on directed and weighted graphs with overlapping communities. *Physical Review E*, 80, 016118.
- Good, B. H., de Montjoye, Y.-A., & Clauset, A. (2010). Performance of modularity maximization in practical contexts. *Physical Review E*, 81, 046106.
- Traag, V. A., Van Dooren, P., & Nesterov, Y. (2011). Narrow scope for resolution-limit-free community detection. *Physical Review E*, 84, 016114.
- Aldecoa, R., & Marín, I. (2013). Exploring the limits of community detection strategies in complex networks. *Scientific Reports*, 3, 2216.
- Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9, 5233.
- Felipe, L. L., Avrachenkov, K., & Menasche, D. S. (2025). From Leiden to Pleasure Island: The Constant Potts Model for community detection as a hedonic game. arXiv:2509.03834.

Clustering-based topic modeling and graph-based text clustering

- Angelov, D. (2020). Top2Vec: Distributed representations of topics. arXiv:2008.09470.
- Rao, R. N., & Chakraborty, M. (2021). Vec2GC -A graph based clustering method for text representations. arXiv:2104.09439.

- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05794.
- Zhang, Z., Fang, M., Chen, L., & Namazi-Rad, M.-R. (2022). Is neural topic modelling better than clustering? An empirical study on clustering with contextual embeddings for topics. *Proceedings of NAACL*. (CETopic)
- Zhang, L., Liu, J., & Yan, Q. (2023). Graph2Topic: An opensource topic modeling framework based on sentence embedding and community detection. arXiv:2304.06653. (G2T)
- Eichin, F., Schuster, M., Groh, G., & Hedderich, M. A. (2024). Semantic Component Analysis: Discovering patterns in short texts beyond topics. arXiv:2410.21054.
- Janssens, W., Bogaert, M., & Van den Poel, D. (2025). LLM-assisted topic reduction for BERTopic on social media data. arXiv:2509.19365.

Hierarchical clustering, RAG and GraphRAG

- Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *ICLR*. arXiv:2401.18059.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A Graph RAG approach to query-focused summarization. arXiv:2404.16130.
- Xiao, Y., Dong, J., Zhou, C., Dong, S., Zhang, Q.-w., Yin, D., Sun, X., & Huang, X. (2025). GraphRAG-Bench: Challenging domain-specific reasoning for evaluating graph retrieval-augmented generation. arXiv:2506.02404.
- Han, H., Ma, L., Wang, Y., Shomer, H., Lei, Y., Qi, Z., Guo, K., Hua, Z., Long, B., Liu, H., Aggarwal, C. C., & Tang, J. (2026). RAG vs. GraphRAG: A systematic evaluation and key insights. arXiv:2502.11371.
- Hossain, J., & Saryüce, A. E. (2026). Core-based hierarchies for efficient GraphRAG. arXiv:2603.05207.

Context and memory compression

- Bohdal, O., Saha, P., Michieli, U., Ozay, M., & Ceritli, T. (2026). Clustering-driven memory compression for on-device large language models. arXiv:2601.17443.
- Zhou, Y., Lei, Y., Si, S., Sun, Q., Wang, W., Wu, Y., Wen, H., Chen, G., Qi, F., & Sun, M. (2025). From context to EDUs: Faithful and structured context compression via Elementary Discourse Unit decomposition. arXiv:2512.14244.

Reference implementations

- `mainlp/semantic_components` (MIT, 2024) –SCA reference implementation by F. Eichin
- `bandeerun/pyhercules` (MIT, 2025) –HERCULES Python implementation (no peer-reviewed publication at the time of this work)

Project artifact

- LayerForge: <https://github.com/ChaiCroquis/LayerForge> (MIT licensed)
- Commit hash for measurements in this paper: see `git log` at submission time
- All CSVs, JSONs, PNGs in `scripts/k_sweep/` are reproducible from the corresponding `*.py` scripts.

Appendix A –Related work summary table

§1.2 (Related work and positioning) で詳述した文献の参照 table。本 appendix は §1.2 の補足であり、各文献の詳細位置付けは §1.2 を参照。

Group (§1.2 番号)	主要文献
Clustering-based topic modeling and graph-based text clustering (§1.2.1)	BERTopic (Grootendorst 2022), Top2Vec (Angelo 2023)

Group (§1.2 番号)	主要文献
Hierarchical RAG and GraphRAG (§1.2.2)	RAPTOR (Sarathi et al. 2024), GraphRAG (Edg
Context and memory compression (§1.2.3)	Clustering-driven Memory Compression (I
Community detection method comparison (§1.2.4)	Lancichinetti & Fortunato (2009), Aldecoa & M

LayerForge の貢献は (i) 既知技術 (G2T-like substrate + SCA + HERCULES + 4±1 + Mode A/B/C) の組合せ実装の報告、(ii) Newman vs CPM の small-N text domain における systematic comparison + 3 failure mode 分類 の 2 点に絞られる。community detection method 比較そのものの新規性は主張せず、本 domain での実証データを文献空間に提供する report という位置付け。Bohdal et al. (Samsung 2026) との同時期 framing 重複は novelty を否定せず、両研究が独立に同じ problem space を identify した事実として記録する。

Appendix B –Sensitivity analyses summary

GitHub repository の companion document (https://github.com/ChaiCroquis/LayerForge/blob/main/docs/08_empirical_findings.md) §2 以降に詳述した感度分析の主要 outcome のみ集約:

- Embedding model (MiniLM vs mpnet): mpnet が cleaner Q peak、両 method 同方向
- Random seed (42 / 123): K_optimal は安定、Newman Q peak K は seed-invariant
- Edge floor: 0 (default) と median similarity (CPM 用 reference) で above-limit fraction は同様の monotonic
- N (corpus size): K=N_themes tracking は Newman で N 20 から安定、CPM は N に依存せず弱い

Document ends.